



SMART CONTRACT AUDIT

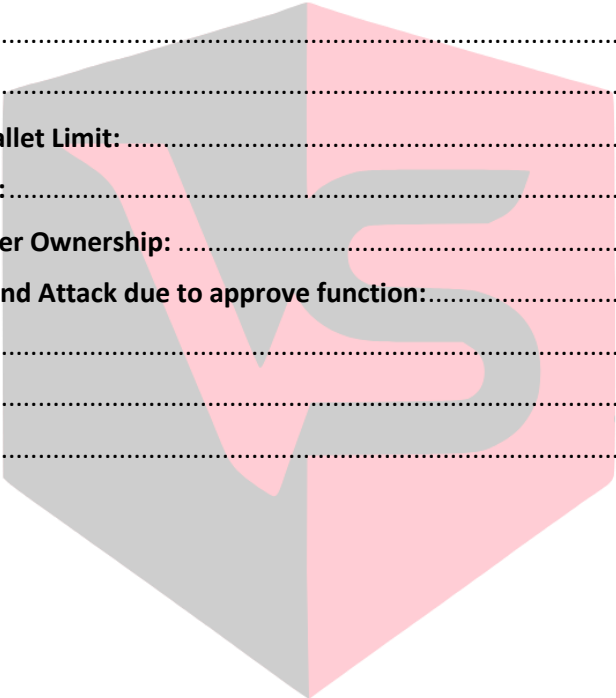
Conducted for: BaitCoin



VULSIGHT

Contents

Executive Summary:	3
Project Background:	3
Audit Scope:	3
Audit Summary:	4
Severity Definitions:	5
Technical Checks:	5
Code Quality and Documentation:	5
Audit Findings:	6
Critical vulnerabilities:	6
High vulnerabilities:	6
Exceeding of Max Wallet Limit:	6
Medium vulnerabilities:	7
No function to transfer Ownership:	7
Potential Double Spend Attack due to approve function:	8
Low vulnerabilities:	8
Disclaimer:	9
Conclusion:	9



VULSIGHT

Executive Summary:

The client contacted our Consulting Firm to conduct a smart contract audit on their Solidity based smart contract. The smart contract wasn't deployed yet on any chain. The contract is a basic ERC-20 token with additional features such as transfer locking and wallet limit functionality. The audit aimed to assess the security, functionality, and overall code quality of the contract. The audit was performed using both manual code analysis and automated security tools. The audit is performed on a best effort basis. Vulnerabilities were detected within the smart contract code during the course of this audit. All of the findings of the audit are detailed inside this report.

Project Background:

The smart contract is written in solidity. The smart contract is a standard implementation of an ERC20 contract with a set of unique features and customizations. This contract doesn't use any libraries.

Audit Scope:

Deployed Address	N/A
Chain	N/A
Language	Solidity
Audit Completion Date	30/7/23

VULSIGHT

Audit Summary:

The primary purpose of this contract is to derive the ERC20 tokens of the contract. The main specifications of the contract was found as follows:

General Information

Contract Symbol	TOK
Contract Decimals	18
Total Supply	100,000,000
Maximum Wallet limit	2,000,000

Functions Summary

- **balanceOf()**: Returns the balance of an address.
- **transfer()**: Transfers the token to a specific address. The function contains checks for ensuring the recipient address is valid, the sender has enough balance, and the transfer amount doesn't exceed the maximum wallet limit (unless the sender is the contract owner).
- **approve()**: Approves a certain amount of tokens to be spent by another address.
- **transferFrom()**: Transfers tokens from one address to another, provided there is enough allowance. This function has similar checks as the transfer() function.
- **allowance()**: Returns the amount of tokens allowed to be spent by a specific address from another address's balance.
- **setTransferEnabled()**: Allows the contract owner to enable or disable transfers.
- **removeWalletLimit()**: Allows the contract owner to remove the maximum wallet limit, effectively setting it to the total supply.

Modifiers

- **onlyOwner**: Only allows the contract owner to call certain functions.
- **onlyUnlocked**: Only allows certain functions to be called when the contract's transfer functionality is enabled or when the sender is the contract owner.

In this review, we analyzed the contract for security risks, code quality, and adherence to best practices. Key areas of consideration included access controls, handling of user funds, and potential reentrancy attacks.

We found:

- **0 Critical risk vulnerabilities**
- **1 High risk vulnerability**
- **2 Medium risk vulnerabilities**
- **0 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Not Passed
Logical Vulnerabilities	Not Passed
Event Emission	Passed
Reentrancy	Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract which was written in Solidity programming language. The code is well indented and clear in nature. While the Code lacked commenting. It is recommended that proper commenting be added to aid developers.

Audit Findings:

Critical vulnerabilities:

0 critical vulnerabilities were found in the smart contract.

High vulnerabilities:

1 high vulnerability was found in the smart contract.

Exceeding of Max Wallet Limit:

The **transfer** and **transferFrom** functions in the contract allow for the transfer of tokens between addresses. They include checks to ensure that the sender has sufficient balance to transfer, that the recipient address is valid, and that the transfer amount does not exceed the defined **maxWalletLimit** (for non-owner addresses).

However, there is a critical omission in these checks - both functions do not check whether the recipient's balance post-transfer would exceed the **maxWalletLimit**. This omission could potentially allow a user's wallet to hold more tokens than the defined **maxWalletLimit**. In this scenario, the sequence of events would be as follows:

1. Address A holds 1.9 million tokens.
2. Address B (not an owner) holds 1.5 million tokens, which is less than the **maxWalletLimit** of 2 million tokens.
3. Address B transfers 1 million tokens to Address A, either directly using **transfer** or indirectly using **transferFrom**.
4. The transfer or **transferFrom** function checks that the transfer value is less than or equal to **maxWalletLimit** or the sender is the owner, which passes as the transfer value is 1 million, which is less than the **maxWalletLimit**.
5. The transfer or **transferFrom** function does not check whether the post-transfer balance of the recipient (Address A) would exceed the **maxWalletLimit**.
6. After the transfer, Address A now holds 2.9 million tokens, which exceeds the **maxWalletLimit**.

The vulnerable code is:

```
function transfer(address _to, uint256 _value) public onlyUnlocked returns (bool) {
    require(_to != address(0), "Invalid recipient address.");
    require(_value <= balances[msg.sender], "Insufficient balance.");
    require(
        _value <= maxWalletLimit || msg.sender == owner,
        "Transfer amount exceeds wallet limit."
    );
}
```

```
        balances[msg.sender] -= _value;
        balances[_to] += _value;

        emit Transfer(msg.sender, _to, _value);
        return true;
    }

    function transferFrom(
        address _from,
        address _to,
        uint256 _value
    ) public onlyUnlocked returns (bool) {
        require(_to != address(0), "Invalid recipient address.");
        require(_value <= balances[_from], "Insufficient balance.");
        require(_value <= allowed[_from][msg.sender], "Insufficient allowance.");
        require(
            _value <= maxWalletLimit || _from == owner,
            "Transfer amount exceeds wallet limit."
        );

        balances[_from] -= _value;
        balances[_to] += _value;
        allowed[_from][msg.sender] -= _value;

        emit Transfer(_from, _to, _value);
        return true;
    }
}
```

VULSIGHT

Medium vulnerabilities:

2 medium vulnerabilities were found in the smart contract.

No function to transfer Ownership:

The smart contract, as currently designed, has an inherent vulnerability due to its lack of functionality for transferring ownership. The contract assigns ownership upon creation, assigning this status to the address that deploys the contract. This ownership role is permanent and irrevocable, with no provision included for transferring it to another address.

This lack of ownership transfer capability presents potential operational risks and challenges:

- **Single Point of Failure:** Should the private key for the owner's address become lost or compromised, it will be impossible to perform administrative actions requiring owner permissions, rendering the contract effectively frozen and unusable.

- **Inability to Transfer Control:** If the original owner wishes to step down, retire, or otherwise hand over control of the contract to another party for any reason, they are unable to do so. This limits the lifecycle and adaptability of the contract and can pose significant business continuity risks.

Potential Double Spend Attack due to approve function:

The **approve** function in the contract allows an owner of the tokens to approve another address (spender) to spend a certain amount of tokens on their behalf. However, the implementation of this function potentially exposes the contract to a double spending attack, also known as a "race condition".

The race condition happens when the token owner calls the **approve** function to change the allowance of a spender.

However, if the spender is aware of the transaction in the pending transaction pool (mempool) that sets their allowance to a new value, they can front-run this transaction with a **transferFrom** transaction with a higher gas price, thereby spending the old allowance. The owner's transaction will then execute, setting the allowance to the desired value. The spender can then spend this allowance as well, effectively double-spending the owner's tokens. The vulnerable code is:

```
function approve(address _spender, uint256 _value) public onlyUnlocked
returns (bool) {
    require(_spender != address(0), "Invalid spender address.");

    allowed[msg.sender][_spender] = _value;

    emit Approval(msg.sender, _spender, _value);
    return true;
}
```

Steps to Reproduce:

1. Address A holds 1 million tokens.
2. Address A approves Address B to spend 500,000 tokens.
3. Address A decides to change the approval to 200,000 tokens, so it sends a transaction to set the allowance to 200,000.
4. Address B, monitoring the mempool, sees this pending transaction.
5. Address B quickly sends a transferFrom transaction with a higher gas price, spending the initially approved 500,000 tokens to Address C.
6. Address A's transaction is then mined, setting the approval to 200,000 tokens.
7. Address B can now spend an additional 200,000 tokens, leading to a total spending of 700,000 tokens.

Low vulnerabilities:

0 low vulnerabilities were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. Vulnerabilities were discovered within it. It is recommended that these vulnerabilities be resolved and additional security measures be undertaken to ensure future security such that of a creation of a bug bounty program.

